

# 分布式消息队列 Kafka

Grissom | 2025年11月

# 目录 >

## CONTENTS

- 1 Kafka简介
- 2 Kafka原理
- 3 Kafka使用



# 1 chapter

## Kafka简介

- ✓ 什么是Kafka
- ✓ 应用场景

### ➤ 概念

- 基于发布/订阅的分布式消息系统
- 由Linkedin开发，用Scala编写

### ➤ 特性

- 消息持久化：采用时间复杂度 $O(1)$ 的磁盘存储结构，即使TB级以上数据也能保证常数时间的访问速度
- 高吞吐：即使在廉价的商用机器上，也能达到单机每秒10万条消息的传输
- 高容错：多分区多副本
- 易扩展：新增机器集群无需停机，自动感知
- 同时支持实时和离线数据处理



### ➤ 异步通信

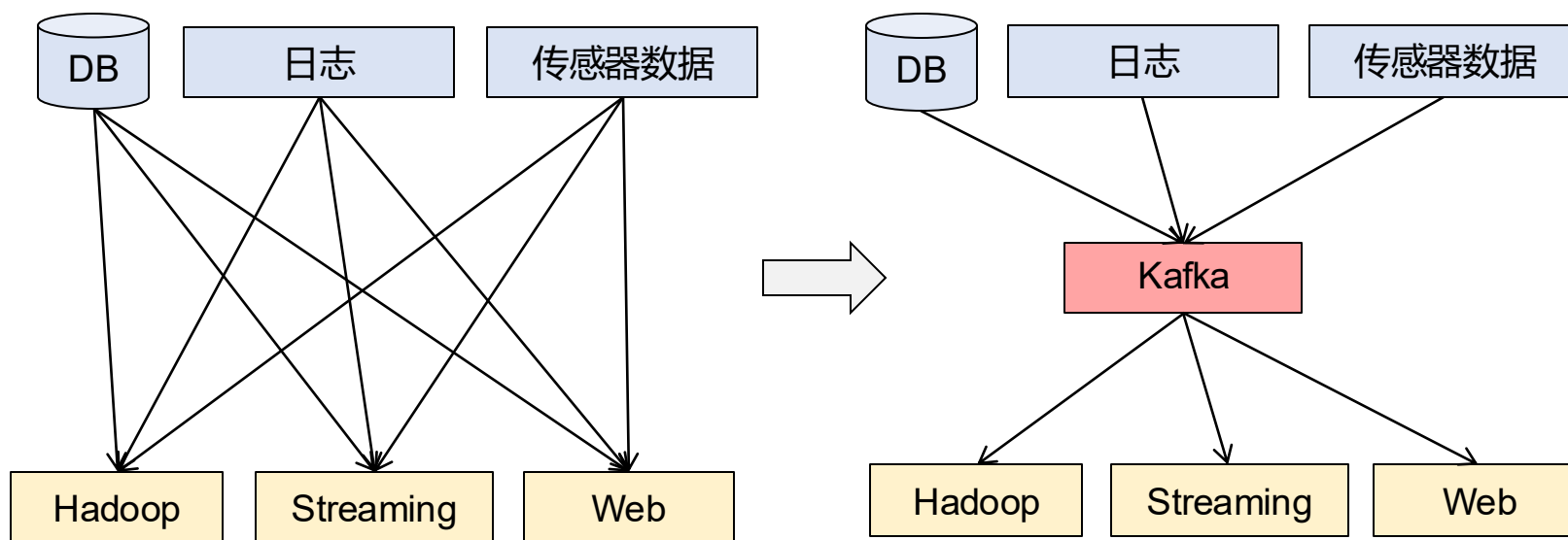
- 将消息放入Kafka，可以不立即处理

### ➤ 应用解耦

- 在应用处理过程中插入一个隐含的、基于数据的接口层

### ➤ 流量消峰

- 关键应用能够顶住访问峰值，不会因超出负荷而崩溃





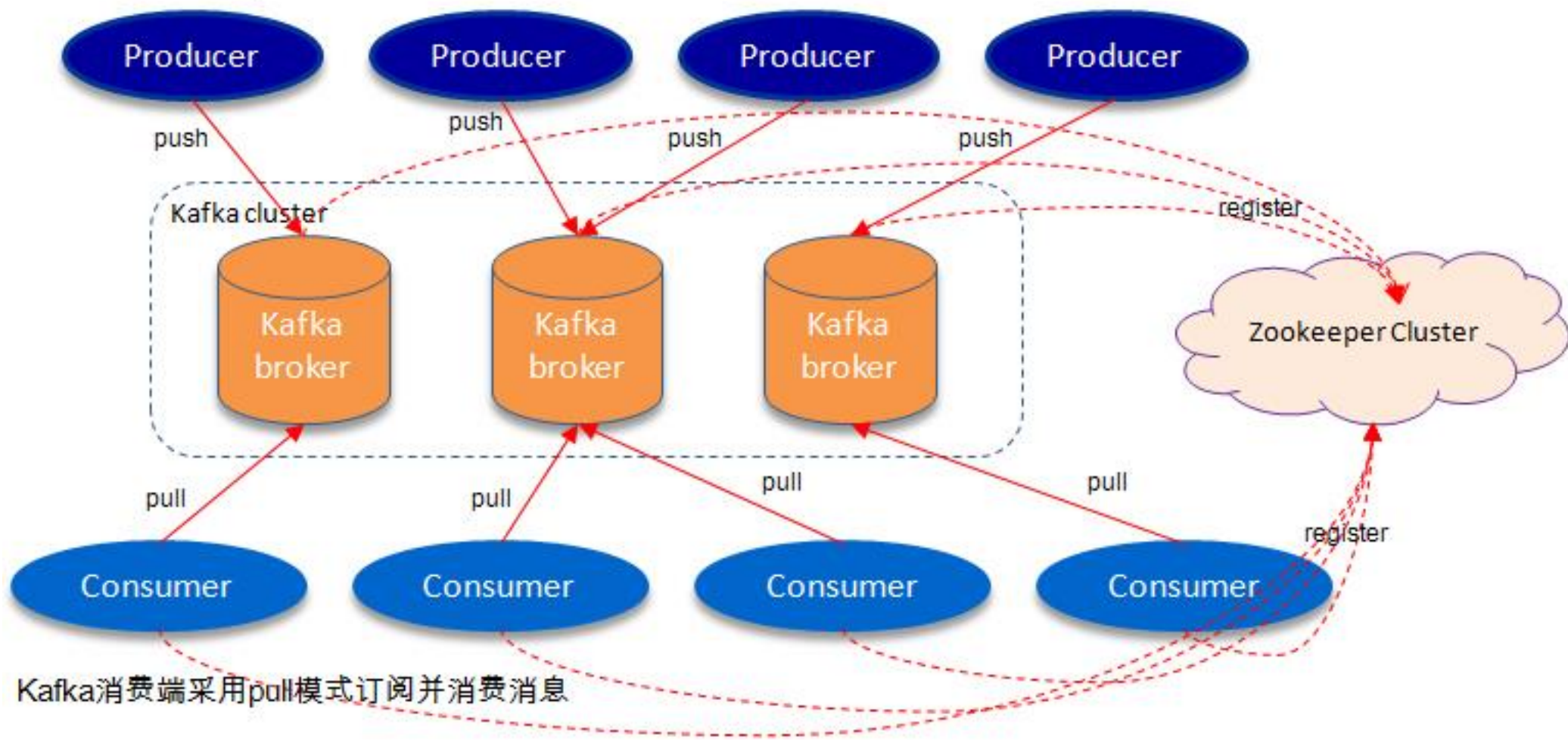
# 2

chapter

## Kafka原理

- ✓ 基本概念
- ✓ 数据存储
- ✓ 高可用

Kafka发送端采用push模式将消息发送到broker



### ➤ Broker (代理)

- Kafka的一个实例或节点，一个或多个Broker组成Kafka集群

### ➤ Producer (消息生产者)

- 向Broker发布消息的客户端

### ➤ Consumer (消息消费者)

- 从Broker消费消息的客户端

### ➤ Zookeeper

- Kafka将主要的元数据保存在Zookeeper中
- 负责Kafka集群管理，包括配置管理、动态扩展、Broker负载均衡、Leader选举等



### ➤ Topic (主题)

- Topic是Kafka中**同一类数据的集合**（贴同一标签），相当于数据库中的表
- Producer将同一类数据写入同一个Topic，Consumer从同一个Topic中读取同类数据
- Topic是**逻辑概念**，只需指定Topic就可以生产或消费数据，不必关心数据的物理存储

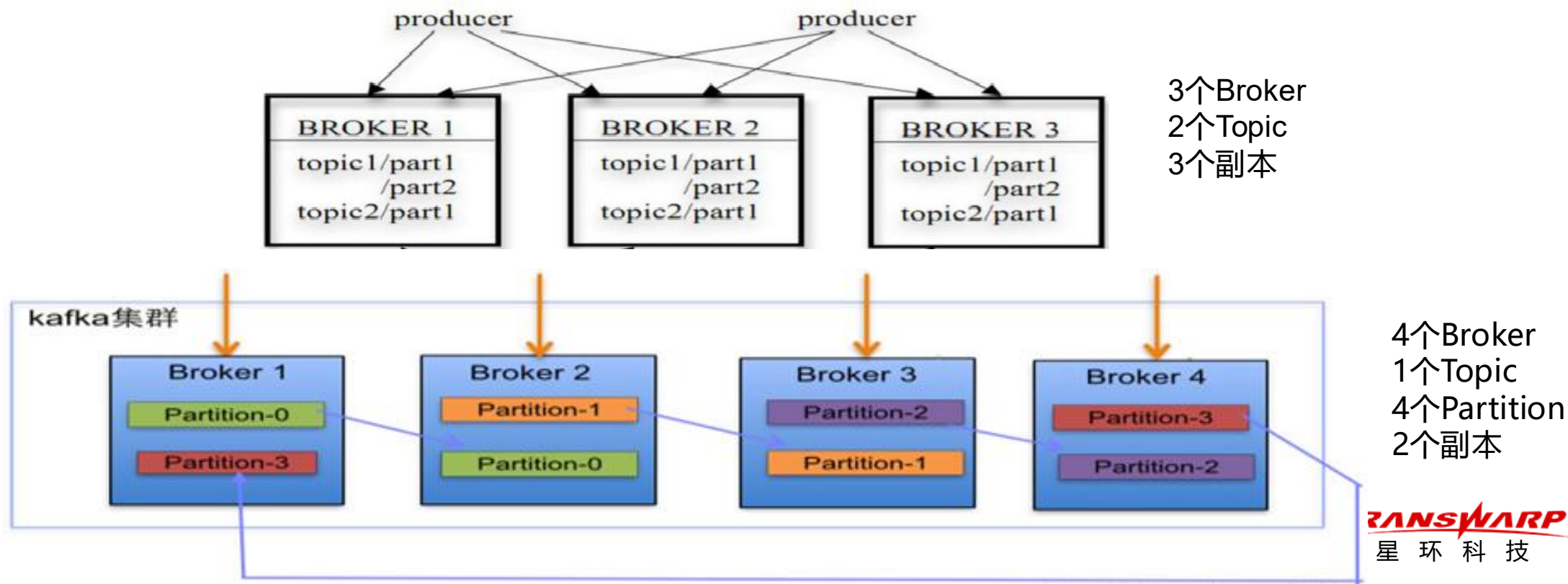
### ➤ Partition (分区)

- 分区是一个有序的、不可修改的消息队列，**分区内消息有序存储**
- 一个Topic可分为多个分区，相当于把一个数据集分成多份，分别存储不同的分区中
- Partition是**物理概念**，每个分区对应一个文件夹，存储分区的数据和索引文件

### ➤ Replication (副本)

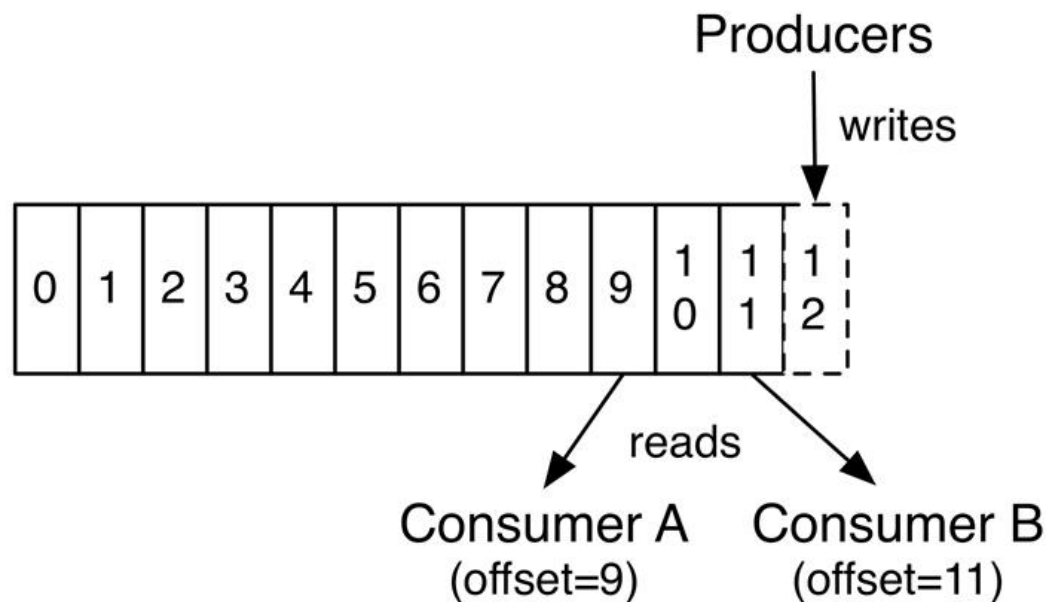
- 一个分区可以设置多个副本，副本存储在不同的Broker中

- 消息在Broker中按Topic（主题）进行分类，相当于为每个消息打个标签
- 一个Topic可分为多个Partition（分区）
- 每个Partition可以有多个Replication（副本）
- 消息存储在Broker的某一Topic的某一Partition中，同时存在多个副本



### ➤ Offset (偏移量)

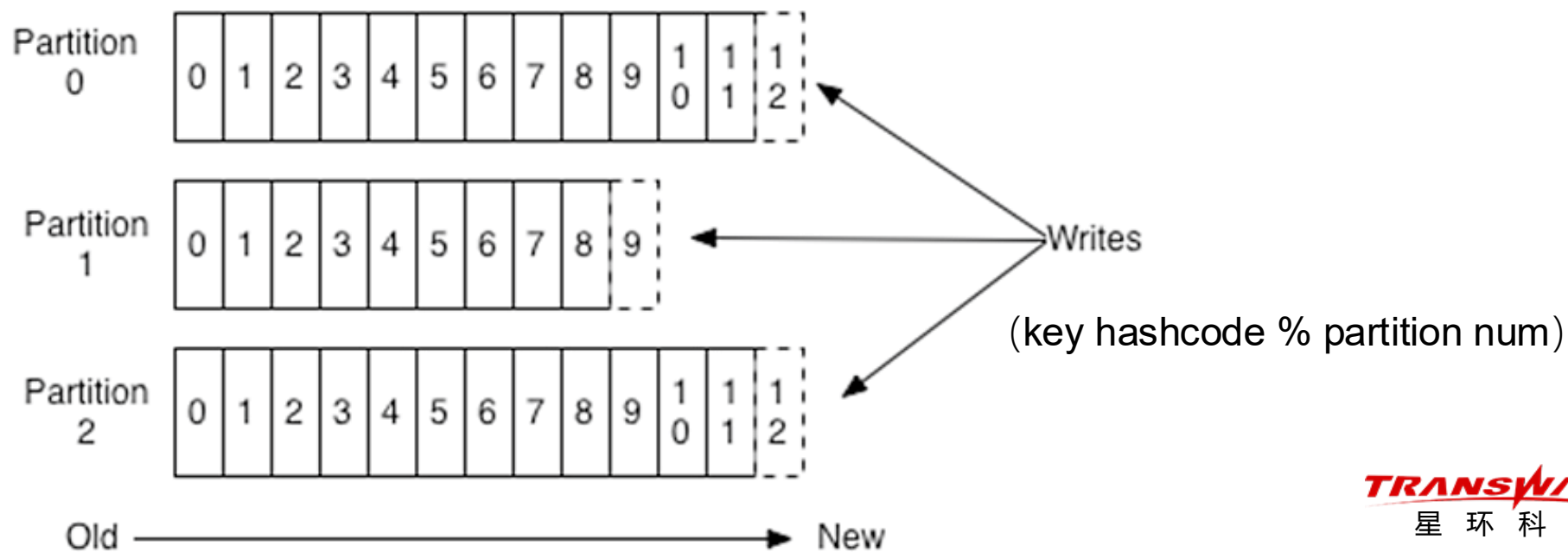
- Offset是定位分区中消息的顺序编号
- Offset用于在分区中唯一标识消息
- 推荐由Kafka（而不是Zookeeper）来存储和维护Offset，即在Broker中记录和重置每个Consumer读取分区消息的偏移量
  - 也可以选择由Zookeeper来存储和维护，但效率会有一定影响



## ➤ Partition (分区)

- Partition是一个FIFO队列，发布消息采用在队列尾部追加的方式，消费消息采用在队列头部顺序读取的方式
- 一个Topic可分为多个Partition，仅保证同一分区内消息有序存储，不保证Topic整体（多个分区之间）有序

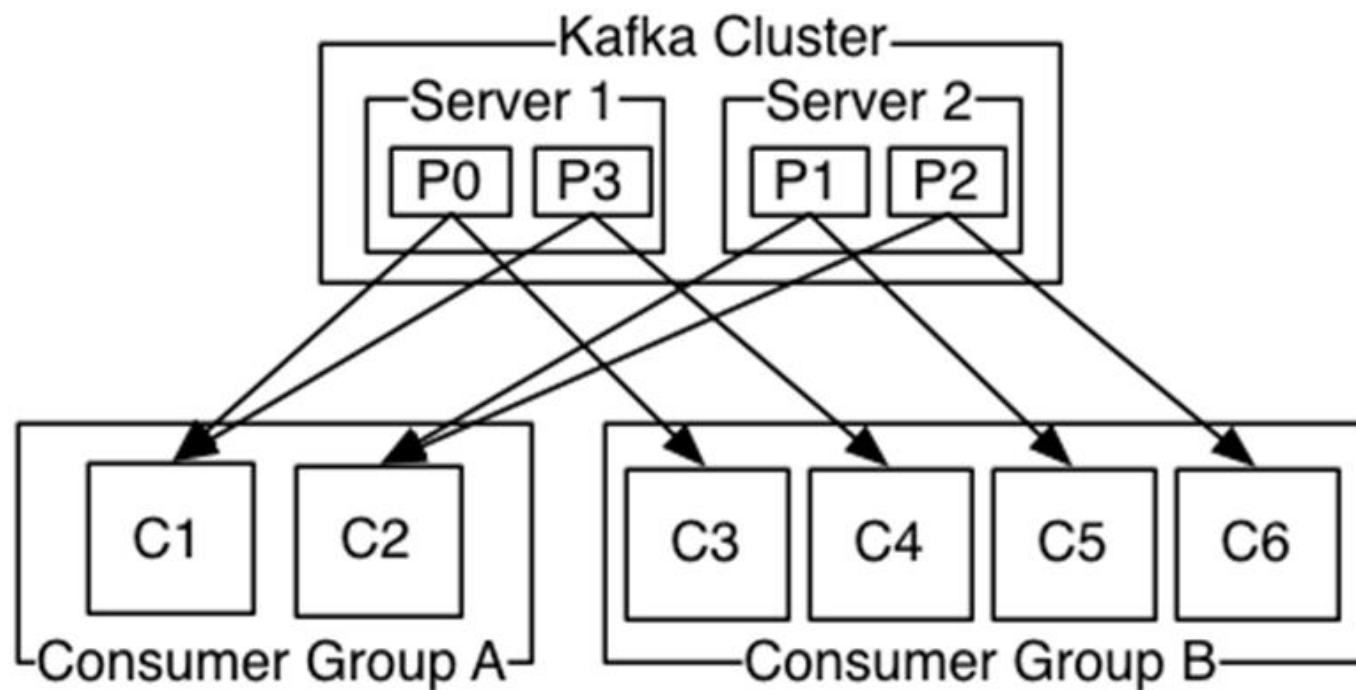
### Anatomy of a Topic





### ➤ Consumer Group (CG, 消费者组)

- 每个Consumer都属于一个特定的CG
- 为了加快读取速度，多个Consumer可划分为一个消费者组，并行消费同一个Topic
- 对于Topic中的某一消息，不同的CG可以重复读取该消息（**组间共享**），但一个CG中只能有一个Consumer可以读取该消息（**组内互斥**）



- 每个Partition副本都是一个目录，目录中包含若干Segment文件

```
drwxrwxr-x. 2 hadoop hadoop 4096 1月 4 09:57 topictest1-0
drwxrwxr-x. 2 hadoop hadoop 4096 1月 4 09:57 topictest1-1
drwxrwxr-x. 2 hadoop hadoop 4096 1月 4 09:57 topictest1-2
drwxrwxr-x. 2 hadoop hadoop 4096 1月 4 09:58 topictest1-3
drwxrwxr-x. 2 hadoop hadoop 4096 1月 4 09:58 topictest1-4
```

- Segment（段文件）

- Segment文件是Kafka的最小数据存储单元，一个Partition包含多个Segment文件
- Segment文件由以消息在Partition中的起始偏移量命名的数据文件（\*.log）和索引文件（\*.index、\*.timeindex）组成

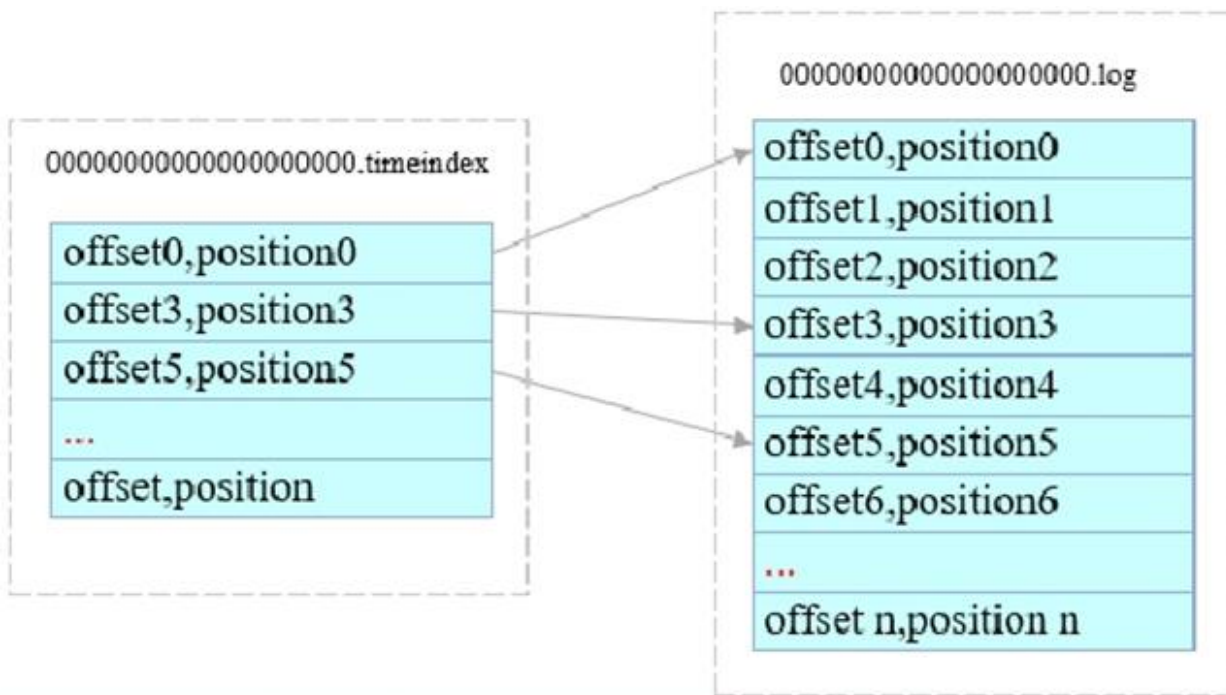
```
000000000000000000000000.log
000000000000000000000000.index
000000000000000000000000.timeindex
00000000000000000000170410.log
00000000000000000000170410.index
00000000000000000000170410.timeindex
```

## ► Kafka索引

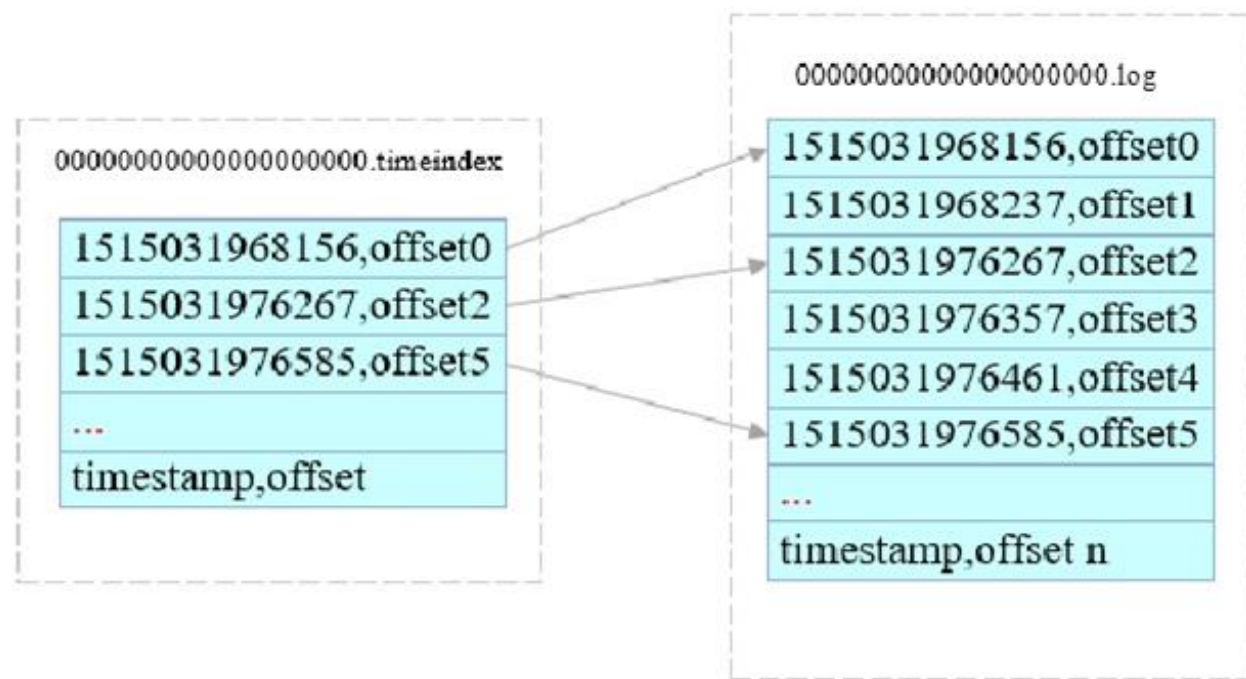
- 为了提高消息写入和查询速度，为每个Partition创建索引，索引文件存储在Partition文件夹下
- 偏移量索引
  - 文件以offset偏移量为名称，以index为后缀
  - 索引数据格式：<offset, position>
  - 采用稀疏存储方式
- 时间戳索引
  - 文件以timeindex为后缀
  - 索引数据格式：<timestamp, offset>
  - 采用稀疏存储方式

```
0000000000000000000000000000.index  
0000000000000000000000000000.log  
0000000000000000000000000000.timeindex
```

## ➤ Kafka索引



偏移量索引



时间戳索引

### ➤ 多分区多副本

- Kafka早期版本没有Replication概念，一旦某个Broker宕机，其上的分区数据就可能丢失
- 一个Topic可以有多个分区，每个分区可以有多个副本，副本存储在不同的Broker中
- 从一个分区的多个副本中选举一个Partition Leader，由Leader负责读写，其他副本作为Follower从Leader同步消息

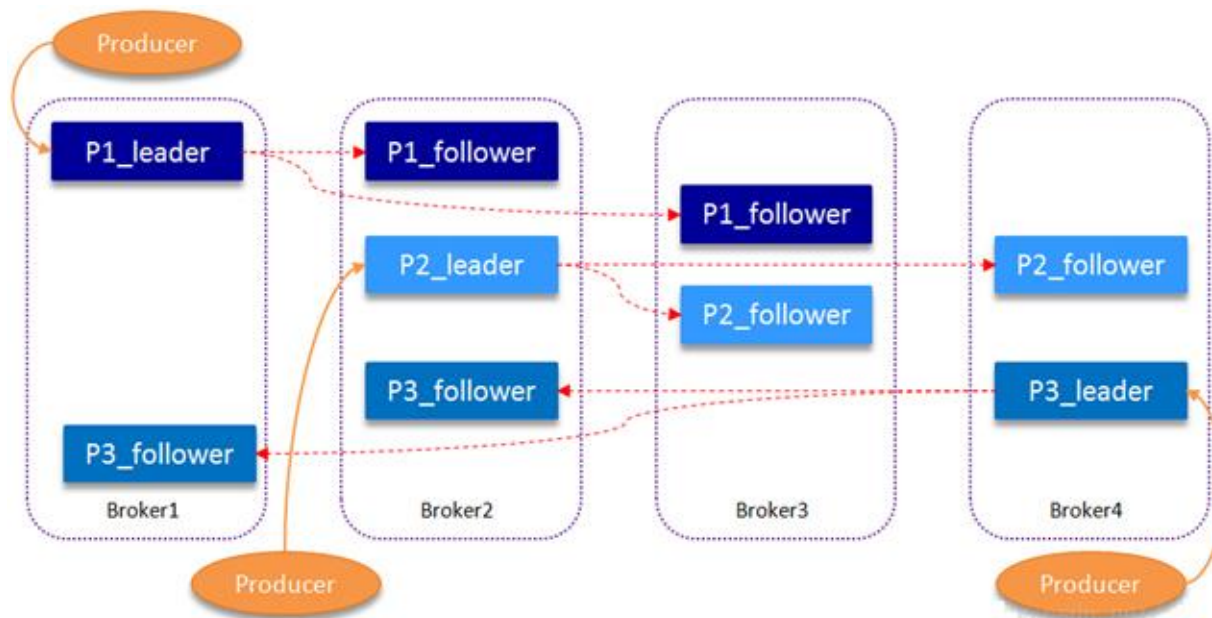
### ➤ Kafka Controller Leader选举

- 每个Broker启动时都会创建一个Kafka Controller进程
- 通过Zookeeper，从Kafka集群中选举出一个Broker作为Kafka Controller Leader
- Kafka Controller Leader负责管理Kafka集群的分区和副本状态，避免分区副本直接在Zookeeper上注册Watcher和竞争创建临时Znode，导致Zookeeper集群负载过重



### ➤ Kafka Partition Leader选举

- Kafka Controller Leader负责Partition Leader的选举
- ISR列表（In Sync Replica）
  - ISR是Zookeeper中的候选副本同步列表，负责保存候选副本（Partition Follower）的状态信息
  - Partition Leader负责跟踪和维护ISR
  - Partition Follower定期从Leader同步数据，若Follower心跳超时或消息落后太多，将被移除出ISR
- Partition Leader挂掉后，Kafka Controller Leader从ISR中选择一个Follower作为新的Leader





# 3 chapter

## Kafka使用

- ✓ Kafka 命令
- ✓ Kakfa API

### ➤ 创建Topic

- 可指定Partition、Replication数量
- 建议--replication-factor > 1，否则Broker宕掉后将无法写入消息

```
# ./kafka-topics.sh
--create --topic demo
--zookeeper tdh-203:2181, tdh-204:2181, tdh-205:2181
--partitions 3 --replication-factor 3
```

### ➤ 查看Topic详情

```
# ./kafka-topics.sh
--describe --topic demo
--zookeeper tdh-203:2181
```

### ➤ 查看Topic列表

```
# ./kafka-topics.sh  
--list  
--zookeeper tdh-203:2181, tdh-204:2181, tdh-205:2181
```

### ➤ 创建Producer

```
# ./kafka-console-producer.sh  
--broker-list tdh-203:9092, tdh-204:9092, tdh-205:9092  
--topic demo
```

### ➤ 创建Consumer

```
# ./kafka-console-consumer.sh  
--bootstrap-server tdh-203:9092,tdh-204:9092,tdh-205:9092  
--topic demo
```

### ➤ 重置Offset

- 含义：当消息消费出现异常时，将offset设置为某个值或最小值，从分区的Offset处重新读取
- 重置前

– logSize为分区消息总数，Lag为未消费消息数，Offset为已消费消息数

```
./kafka-run-class.sh kafka.tools.ConsumerOffsetChecker --zookeeper tdh-203:2181 -g group1 --topic demo1
```

| Group  | Topic  | Pid | Offset | logSize | Lag | Owner |
|--------|--------|-----|--------|---------|-----|-------|
| group1 | topic1 | 0   | 3      | 3       | 0   | none  |

- 重置

```
./kafka-run-class.sh kafka.tools.UpdateOffsetsInZK earliest ../config/consumer.properties demo
```

- 重置后

```
./kafka-run-class.sh kafka.tools.ConsumerOffsetChecker --zookeeper tdh-203:2181 -g group1 --topic demo1
```

| Group  | Topic  | Pid | Offset | logSize | Lag | Owner |
|--------|--------|-----|--------|---------|-----|-------|
| group1 | topic1 | 0   | 0      | 3       | 3   | none  |



### ➤ Producer API:

```
import org.apache.kafka.clients.producer.*;
public class CustomProducer {
    public static void main(String[] args) throws ExecutionException, InterruptedException {

        Properties props = new Properties();
        props.put("bootstrap.servers", "hadoop1:9092");//kafka集群, broker-list
        props.put("acks", "all");
        props.put("retries", 1);//重试次数
        props.put("batch.size", 16384);//批次大小
        props.put("linger.ms", 1);//等待时间
        props.put("buffer.memory", 33554432);//RecordAccumulator缓冲区大小
        props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

        Producer<String, String> producer = new KafkaProducer<>(props);
        for (int i = 0; i < 100; i++) {
            producer.send(new ProducerRecord<String, String>("first", Integer.toString(i), Integer.toString(i))); }
        producer.close();
    }
}
```

## ➤ Consumer API:

```
import org.apache.kafka.clients.consumer.*;

public class CustomConsumer {
    public static void main(String[] args) {

        Properties props = new Properties();
        props.put("bootstrap.servers", "hadoop1:9092");
        props.put("group.id", "test");//消费者组，只要group.id相同，就属于同一个消费者组
        props.put("enable.auto.commit", "false");//自动提交offset

        props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Arrays.asList("first"));
        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(100);
            for (ConsumerRecord<String, String> record : records) {
                System.out.printf("offset = %d, key = %s, value = %s%n", record.offset(), record.key(), record.value()); }
            consumer.commitSync();
        }
    }
}
```



# Q&A

**TRANSWARP**  
星环科技

## 温故知新

- 为什么要对Consumer进行分组?
- 为什么Kafka分了Topic之后，还要分Partition?
- 请简述Kafka的索引机制。
- Partition Leader和Follower是如何分工合作的?
- 为什么Zookeeper不亲自负责Partition Leader选举?
- 请简述Kafka的高可用机制。

